Peptide Binder Design with AfDesign

Instructor guide — preparation, delivery and known issues

Keith Fraser, PhD · Senior Lecturer, Department of Biological Sciences · Rensselaer Polytechnic Institute
`online/Peptide Binder Design.ipynb`

At a glance

Runtime	**GPU, mandatory.** Runtime → Change runtime type → T4 GPU. On CPU the design loop is unusable.
Wall clock	8–15 minutes for the first run, most of it the one-time setup. Budget 60–75 minutes.
Network	Heavy. Installs ColabDesign from GitHub and downloads the AlphaFold parameter archive — **5.6 GB** — on every fresh session.
Colab only	It imports `google.colab` and uses `files.upload()`. It will not run in Jupyter, VS Code or locally.
Prerequisites	The structure-prediction session, for pLDDT and what AlphaFold is doing.
Produces	`binder.pdb` (named after the protocol), an animation, a PSSM heat map
Quiz items	**0.** This notebook ships with no quizzes, no objectives and no citations.

What this notebook is

This is Sergey Ovchinnikov's AfDesign binder notebook from the ColabDesign repository, pinned at `v1.1.1`, unmodified. It is not one of my builds, and it does not follow the structure of the rest of the set — no numbered sections, no formative questions, no references. I use it as a capstone demonstration rather than as a worksheet, and this guide is written accordingly.

What it does: given a target protein structure and a binder length, it runs gradient-based optimisation *through* AlphaFold, searching sequence space for a peptide that AlphaFold predicts will fold against the target with many interface contacts and high binder pLDDT. That is hallucination — the sequence is invented by inverting the predictor.

The notebook's own header carries a warning worth reading aloud to the class:

> Using AfDesign as the only "loss" function for design might be a bad idea, you may find adversarial sequences (aka. sequences that trick AlphaFold).

The target, and why it is a good one

The default is **PDB 4N5T, chain A** — the 1.7 Å structure of MDMX (Mdm4, zebrafish) bound to the stapled α-helical peptide **ATSP-7041**. That is not an arbitrary choice and it is the pedagogical gift in this notebook.

MDMX and its paralogue MDM2 bind and inactivate p53. Blocking that interaction reactivates p53, which is why it has been a cancer drug target for twenty years. The p53 transactivation helix binds through three hydrophobic residues — Phe19, Trp23 and Leu26 — that insert into three pockets on the MDM2/MDMX

surface. ATSP-7041 is an engineered stapled peptide that reproduces that pharmacophore with a hydrocarbon staple locking the helix.

So when the notebook hallucinates a 14-residue binder against chain A, there is a known right answer in the crystal structure it is docking against. That makes it assessable:

- Does the designed sequence come out helical?
- Does it place large hydrophobics — Phe, Trp, Leu — at i, i+4, i+7 spacing?
- Do those residues land in the same three pockets ATSP-7041 occupies?

Students who get a helix with an F/W/L face have reproduced a real pharmacophore from scratch, and they can see it by overlaying their output on the deposited peptide. Students who get a high-scoring sequence that does none of this have found an adversarial example, which is the other half of the lesson.

Before class

Request the GPU runtime first. Then run the setup cell and go get coffee: it pip-installs ColabDesign from GitHub and pulls `alphafold_params_2022-12-06.tar`, which is **5.6 GB**. The download uses `aria2c -x 16` so it is usually a few minutes on Colab's network, but it happens again every time the session is recycled. If you are running a live session, have everyone start that cell at the very beginning while you talk through the biology.

Two failure modes to expect:

- **No GPU allocated.** Free Colab will sometimes refuse. The notebook will still run on CPU and the optimisation will take hours. Check `Runtime → Change runtime type` shows T4 and, if you want certainty, have students run `!nvidia-smi` in a scratch cell first.
- **Disk full.** The parameter archive plus the extracted copy is over 11 GB. Colab's standard disk handles it; a session already carrying other data may not.

The form cells

Every cell is a Colab form (`#@title`, `#@param`), so students see widgets rather than code. The parameters worth explaining:

PARAMETER	DEFAULT	WHAT TO SAY
`pdb`	`4N5T`	PDB code, UniProt accession (fetches the AlphaFold DB model), or blank to upload
`target_chain`	`A`	Chain A is MDMX
`target_hotspot`	empty	Restricts the interface loss to named positions. Setting this to the p53-binding cleft residues is the single most useful experiment in the session
`binder_len`	`14`	ATSP-7041 is 14 residues. Not a coincidence
`optimizer`	`pssm_semigreedy`	120 gradient steps to build a position-specific profile, then 32 semi-greedy mutation steps
`num_models`	`2`	More models means slower and less overfitting to one network
`num_recycles`	`0`	Raise to 3 for better geometry at roughly 3× the cost

The `{run: "auto"}` tag on the display cell means it re-executes whenever a widget changes. Warn students: clicking a dropdown re-runs the cell. It is a common source of confusion.

Known issues in this version

- **The PSSM heat map is normalised along the wrong axis under the default optimizer.** In the `pssm_semigreedy` branch the code is `softmax(model._tmp["seq_logits"], 1)`; every other branch in the same cell uses `-1`. Axis 1 is the position axis, so probabilities sum to one across the 14 positions instead of across the 20 amino acids. The plot in the last cell is then scaled to `zmax=1` and looks washed out, with no column summing to 1. One-character fix: change the `1` to `-1`. Since `pssm_semigreedy` is the default and the recommended setting, every student hits this.
- **No validation step.** The notebook reports what AlphaFold thinks. There is no independent check — no Rosetta rescoring, no MD, no ProteinMPNN redesign of the output backbone, and obviously no experiment. Published success rates for hallucinated binders taken straight to the bench are low; Bennett et al. (2023) is the reference for how much machinery it takes to raise them.
- **Non-determinism.** Different runs give different sequences. That is the nature of the method, not a bug, but tell students in advance or they will think they did something wrong. Have them run it two or three times and compare — the *consensus* across runs is far more informative than any single output.
- **No objectives, quizzes or citations.** If you need this session to match the assessment pattern of the others, those have to be added. The four learning objectives I would state are below.

Objectives I state at the start

1. Explain what hallucination means in protein design, and how it differs from inverse folding.
2. Run a gradient-based design loop through a structure predictor and read its loss trace.
3. Interpret a designed sequence's PSSM and relate it to the physical chemistry of the target interface.
4. Explain why a high predicted confidence is not evidence of binding, and name two independent checks you would run before ordering the peptide.

Discussion I run while the optimiser works

The design loop takes a few minutes, which is the right amount of time for the question the whole session exists to raise: **what has been demonstrated when AlphaFold says yes?**

The optimiser is maximising a quantity produced by a model that was trained to reproduce the PDB. A sequence that maximises it may be a good binder, or it may be an input that exploits a blind spot — the machine-learning term is adversarial example, and the notebook's author says so in the header. The honest position is that hallucination generates hypotheses at a rate no human could, and that each one still needs orthogonal evidence.

Good follow-up questions: what independent computational check would you run first, and why? What would you want to measure experimentally, and what affinity would make the design interesting? If the designed sequence is mostly hydrophobic, is that binding or aggregation?

Further reading I hand out

- Anishchenko et al., *Nature* 600:547 (2021). `10.1038/s41586-021-04184-w` — de novo design by network hallucination.
- Norn et al., *PNAS* 118:e2017228118 (2021). `10.1073/pnas.2017228118` — sequence design by conformational landscape optimisation, the loss family used here.
- Jumper et al., *Nature* 596:583 (2021). `10.1038/s41586-021-03819-2` — AlphaFold2.
- Bennett et al., *Nat Commun* 14:2625 (2023). `10.1038/s41467-023-38328-5` — what it takes to make designed binders work experimentally.
- Chang et al., *PNAS* 110:E3445 (2013). `10.1073/pnas.1303002110` — ATSP-7041, the peptide in the target structure.
- ColabDesign: `github.com/sokrypton/ColabDesign`. No paper; cite the repository and the version tag, `v1.1.1`.